INTERNSHIP PROJECT PHASE (2 WEEKS)

PROJECT GUIDELINES

- **Choose any 1 or 2 projects from the given list.**
- **You are free to improvise — take the given project as a base and modify it as you like.**
- **You can use any tools, technologies, or steps you're comfortable with — there are no restrictions.**
- **Focus and work sincerely so that you have complete clarity and can explain the project confidently in interviews.**
- **Go through the Top 50 Interview Questions for your domain (attached at the end).**
- **Update your project status regularly when the Google Form is shared in group.**
- **while working on the project YOU CAN CHOOSE ANY DATASET RELAVENT TO THE PROJECT.**

.

After project completion, prepare a 1–2 page report in PDF format, containing:

- Introduction
- Abstract
- Tools Used
- Steps Involved in Building the Project
- Conclusion

◆ Note: Report must not exceed 2 pages.

DEAR INTERNS,

YOU HAVE TO UPDATE STATUS OF YOUR PROJECT EVERY 3 OR 4 DAYS ONCE WHEN THE UPDATION LINK IS SHARED IN THE GROUP.

Final submission links will be shared later.

! READ ALL THE GUIDELINES CAREFULLY !

LIST OF PROJECTS

1. Smart Resume Builder with AI Suggestions

Objective: Build a resume generator that suggests improvements using AI (via free APIs).

Tools: React.js, Node.js, OpenAI API (free tier), Tailwind CSS, MongoDB

Mini Guide:

- Design input fields for resume data with React forms. Connect to a
- backend that stores, formats, and exports the resume. Use OpenAI (GPT-
- 3.5 free tier) to suggest improvements based on input. Implement PDF
- export functionality. Add preview mode with print styling.
-

Deliverables:

- Interactive resume builder, PDF export, AI suggestion feature.

2. Developer Portfolio Generator (Markdown to Website)

Objective: Convert Markdown-based resume/portfolio into a customizable website.

Tools: Next.js, Tailwind CSS, Markdown parser, GitHub Pages

Mini Guide:

- Allow users to input .md content.
- Parse Markdown into HTML using a library like remark.
- Design multiple themes and let users switch live.
- Add deploy-to-GitHub functionality.
- Allow preview and customization of sections.

Deliverables:

Web app that converts .md to hosted portfolio site

3.Open Source GitHub Project Explorer

Objective: Build a dashboard to explore trending GitHub repositories with filters.

Tools: React, GitHub API, Chart.js, Tailwind CSS

Mini Guide:

- Fetch data using GitHub REST API.
- Allow sorting by stars, language, update date.
- Add search and tag-based filters.
- Show charts of contributions, issues, etc.
- Include bookmarking and note-taking for repos.

Deliverables: Interactive GitHub repository explorer with analytics.

4. Real-Time Chat App with Socket.IO
Objective: Build a secure, scalable real-time chat platform.
Tools: Node.js, Express, Socket.IO, MongoDB, Tailwind CSS
Mini Guide:

- Set up authentication with sessions or JWT.
- Integrate Socket.IO for message broadcasting.
- Create user groups and private chats.
- Store chat history in MongoDB.
- Add typing indicators and online status.

- **Deliverables:** Working real-time chat app with rooms, private messages, and history.

5. Freelance Bidding Platform (Mini Fiverr Clone)
Objective: Build a service-based freelance job bidding site.
Tools: MERN Stack (MongoDB, Express, React, Node.js)
Mini Guide:

- Create user roles (client/freelancer).
- Implement project posting and bidding system.
- Add messaging and contract acceptance.
- Integrate job tracking and completion approval.
- Use Stripe Test API for mock payments (optional).

Deliverables: Freelance marketplace with job posts, bids, and contracts.

6. Budget Planner with Data Visualizations

- **Objective:** Create a finance dashboard that tracks income, expenses, and goals.

Tools: Vue.js, Chart.js or D3.js, Firebase (free tier)
Mini Guide:

- Design input forms for income/expense entries.
- Visualize data in pie charts, bar graphs.
- Implement filters by category, month.
- Store user data securely using Firebase Auth + Firestore.
- Add savings goals and visual progress indicators.

Deliverables: Interactive budgeting dashboard with analytics and persistence.

7. E-commerce Storefront with Admin Panel

Objective: Build an e-commerce site with inventory management.

Tools: Next.js, MongoDB, Stripe (test mode), Tailwind CSS

Mini Guide:

- Implement product catalog, search, and filtering.
- Add a shopping cart with checkout flow.
- Build a backend for product and order management.
- Include role-based login (admin/user).
- Use Stripe Test Mode for payment simulation.

- Deliverables: Functional e-commerce website with admin dashboard.

8. Habit Tracker Progressive Web App

- Objective: Build a PWA that tracks habits with streaks and reminders.
- Tools: React, IndexedDB (via idb), Workbox (for PWA), Tailwind
- Mini Guide:
  Create local storage using IndexedDB for offline use.
  Track daily check-ins and generate streaks.
  Set up PWA service worker with Workbox.
  Add push notification mock/reminder feature.
  Use charts to show progress over time.
- Deliverables: Installable offline-first habit tracker with analytics.

9. Custom CMS for Blog Publishing

- Objective: Build your own headless CMS to manage blog posts.
- Tools: Express.js, MongoDB, JWT Auth, React, Quill.js
- Mini Guide:
  - Implement a rich text editor (Quill/Slate).
  - Build routes to fetch and post blog content.
  - Admin panel to manage posts, drafts, and categories.
  - Frontend to render dynamic blogs.
  - Optional: Add static export feature.
- Deliverables: Headless CMS backend + React-based blog frontend.

10. SaaS-style Landing Page + Sign Up Workflow

- Objective: Build a SaaS product landing with lead capture and email verification.
- Tools: HTML, Tailwind CSS, Node.js, MongoDB, Nodemailer (free)
- Mini Guide:
  - Create an eye-catching landing page with conversion-focused design.
  - Add sign-up with form validation.
  - Send verification email using Nodemailer.
  - Store leads in MongoDB.
  - Create a thank-you dashboard after sign-up.
- Deliverables: Conversion-optimized landing + full sign-up and email flow.

11. AI-Powered FAQ Chatbot

- Objective: Build a chatbot that answers FAQ using NLP or a small ML model.
- Tools: React, TensorFlow.js or OpenAI API (free tier), Node.js
- Mini Guide:
  - Train a small FAQ model or use keyword matching.
  - Build a chatbot frontend with a messaging UI.
  - Integrate AI logic to parse and respond to questions.
  - Add feedback buttons for each answer.
  - Store unanswered queries for future improvement.
- Deliverables: Interactive chatbot that handles FAQs using ML or AI.

12. Event Management & RSVP System

- Objective: Create a platform to host events and allow users to RSVP.
- Tools: Django, PostgreSQL, Bootstrap, Celery (for email)
- Mini Guide:
  - Set up models for events, users, and invitations.
  - Create RSVP forms and confirmation logic.
  - Add email reminders using Django + Celery + Redis.
  - Create event dashboard with filters.
  - Add QR code check-in system (bonus).
- Deliverables: Full-stack RSVP and event management system.

13. Online Code Editor with Live Preview

- Objective: Create a browser-based IDE for HTML, CSS, and JS.
- Tools: React, Monaco Editor, iframes
- Mini Guide:
  - Embed Monaco Editor for code input.
  - Render code inside an iframe for live preview.
  - Add templates and code sharing via URL or database.
  - Optional: Save snippets to cloud storage.
  - Enable multiple layout views (side-by-side, full preview).
- Deliverables: Web-based code editor with instant preview and sharing.

14. Roadmap Tracker for Career Goals

- Objective: Build a goal-setting and milestone tracking app for tech learners.
- Tools: Svelte, Firebase, Mermaid.js
- Mini Guide:
  - Create UI to add/edit learning goals. Visualize roadmaps
  - with Mermaid.js. Store progress and completion status in
  - Firestore. Include motivation streaks, badges. Add
  - collaboration or mentor-feedback features (optional).
  -
- Deliverables: Personalized career roadmap tracker with goal visualization.

15.Privacy-Focused Notes App with Encryption

- Objective: Build a note-taking app with client-side encryption.
- Tools: React, CryptoJS, IndexedDB
- Mini Guide:
  - Build basic CRUD note interface.
  - Use AES encryption on the client side before storage.
  - Save notes to IndexedDB or localStorage.
  - Add search, pinning, and archive functionality.
  - Optionally integrate cloud sync with optional auth.
- Deliverables: Secure, offline-first note-taking app with encryption.

TOP 50 INTERVIEW QUESTIONS FOR WEB DEVELOPMENT

1. What is the difference betweenid and class in HTML?
2. How do HTML5 semantic elements improve SEO and accessibility?
3. What is the box model in CSS?
4. How do you center a div using Flexbox?
5. What's the difference between relative, absolute, and fixed positioning?
6. What is specificity in CSS and how is it calculated?
7. What are media queries? Give an example.
8. What is the difference between em, rem, px, and % units?
9. How does z-index work in CSS?
10. What are pseudo-elements and pseudo-classes in CSS?
11. What is hoisting in JavaScript?
12. Explain the difference between == and ===.
13. What is a closure? Provide an example.
14. What are var, let, and const? Differences?
15. What is the event loop in JavaScript?
16. Explain how this keyword behaves in different contexts.
17. What is the difference between synchronous and asynchronous code?
18. What are promises and async/await?
19. What is debouncing and throttling?
20. What is the difference between null and undefined?
21. What are React hooks? Name a few and their uses.
22. Explain the difference between controlled and uncontrolled components.
23. What is the virtual DOM and how does React use it?
24. What are props and state in React?
25. How does lifting state up work in React?
26. What are keys in React and why are they important?
27. What is the difference between useEffect and useLayoutEffect?
28. How does React handle reconciliation and re-rendering?
29. What is context API and when would you use it?
30. What are higher-order components (HOCs)?
31. What is Node.js and how is it different from the browser environment?
32. What are middleware functions in Express.js?
33. How does the event-driven model work in Node.js?
34. What are the benefits of using Express.js?
35. How do you handle errors in Node.js?
36. What is the role of package.json?
37. What are the different types of HTTP methods and their use cases?
38. What are streams in Node.js?
39. What is the difference between require and import?
40. How do you manage environment variables in Node.js?
41. What is the difference between SQL and NoSQL databases?
42. What are CRUD operations?
43. What is REST and what are its key principles?
44. What is the difference between PUT and PATCH methods?
45. What are status codes in HTTP and what do 200, 404, 500 mean?
46. How do you secure a REST API?
47. What is CORS and how do you handle it?
48. What is indexing in databases and why is it important?
49. What is JWT and how does it work for authentication?
50. How do you structure a RESTful API?

Elevate Labs